WEB DEVELOPMENT CASE STUDIES

Problem Statement • Objectives • Algorithm • Flowchart • Pseudocode • HTML/CSS Code • Test Cases • Output Screenshots

Case Study 1	E-Learning Website – LearnSphere
Case Study 2	Student Dashboard – EduPortal
Case Study 3	Doctor Appointment Website

Prepared from the three webpage case studies and the nine screenshot pages supplied. Original screenshots are included at the end as output evidence.

CASE STUDY 1: E-Learning Website – LearnSphere

1. Problem Statement

Design a responsive e-learning website that presents the home page, course categories, popular courses, instructor profiles, and footer information. Learners should be able to explore the learning platform clearly.

2. Objectives

• Create navigation and a home-page introduction.

• Display course categories and popular courses in cards.

• Show instructor profiles and footer links.

• Apply consistent colours, spacing, typography, and responsive CSS.

3. Assumptions

• This is a front-end demonstration; login and payment are not connected.

• Course and instructor details are sample content.

4. Algorithm

1. Start.

2. Create the HTML document and navigation menu.

3. Add the hero and course-category sections.

4. Create popular-course and instructor cards.

5. Add footer links and contact information.

6. Apply CSS styling and responsive rules.

7. Open the page in a browser and check the layout.

8. Stop.

5. Flowchart

START → Create structure → Add content → Apply CSS → Test layout → Display webpage → END

6. Pseudocode

```
BEGIN
 CREATE LearnSphere webpage
 DISPLAY navigation and home banner
 DISPLAY course categories
 DISPLAY popular course cards
 DISPLAY instructor profiles
 DISPLAY footer information
 APPLY responsive CSS
 TEST page layout
END
```

7. HTML/CSS Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>LearnSphere</title>
<style>
body{font-family:Arial;margin:0;background:#f4f8ff;color:#142850}
nav,footer{background:#142850;color:white;padding:18px}
main{padding:24px}.cards{display:flex;gap:16px;flex-wrap:wrap}
.card{background:white;padding:18px;border-radius:10px;flex:1 1 200px}
h1,h2{color:#27496d}
</style></head>
<body>
<nav>LearnSphere | Home | Courses | Instructors | Contact</nav>
<main><h1>Learn. Grow. Succeed.</h1>
<p>Explore courses and build new skills.</p>
<h2>Course Categories</h2><section class="cards">
<article class="card">Web Development</article>
<article class="card">Python Programming</article>
<article class="card">Artificial Intelligence</article></section>
<h2>Popular Courses</h2><section class="cards">
<article class="card">HTML &amp;amp; CSS</article>
<article class="card">Python Basics</article>
<article class="card">AI Fundamentals</article></section>
<h2>Instructor Profiles</h2><section class="cards">
<article class="card">Web Instructor</article>
<article class="card">Python Instructor</article></section></main>
<footer>About | Contact | Help</footer></body></html>
```

8. Test Cases

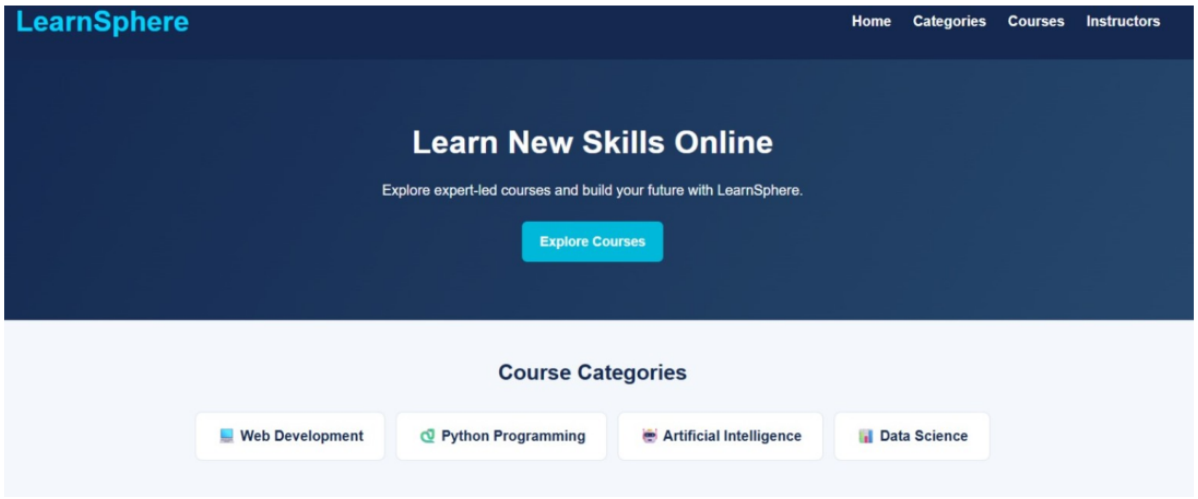
Test	Action / Check	Expected Result
1	Open home page	Navigation and hero section appear
2	View categories	Category cards are visible
3	View popular courses	Course cards are displayed
4	Resize browser	Cards wrap on narrow screens

ne Page and Course Categories

OUTPUT SCREENSHOT 1

Website – LearnSphere

Home Page and Course Categories

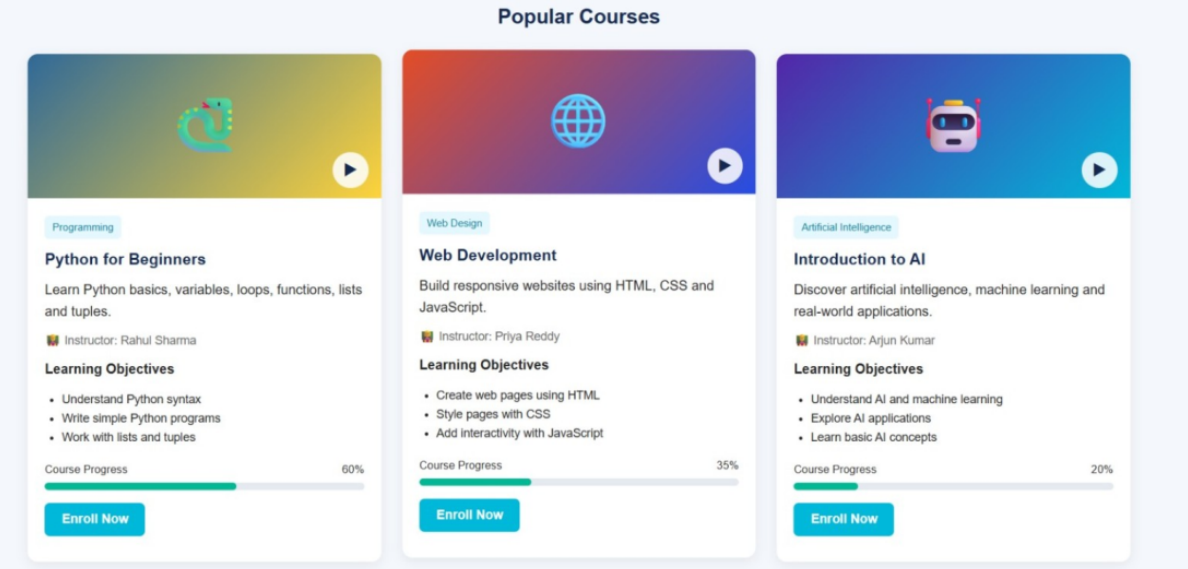


Page

OUTPUT SCREENSHOT 2

Website – LearnSphere

Popular Courses

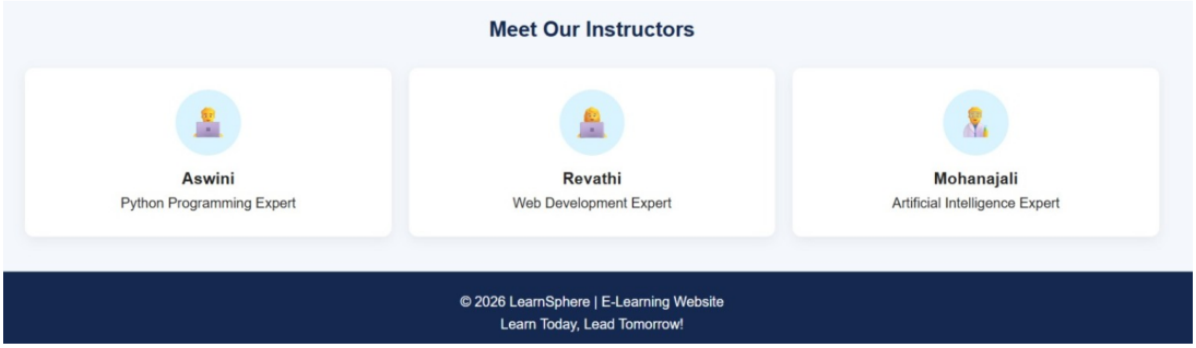


Page

OUTPUT SCREENSHOT 3

Website – LearnSphere

Instructor Profiles and Footer



CASE STUDY 2: Student Dashboard – EduPortal

1. Problem Statement

Create a student dashboard displaying student details, semester results, attendance percentages, subject information, examination schedules, and announcements. Use tables, progress bars, cards, and a sidebar layout.

2. Objectives

• Display student profile and academic overview.

• Show semester results and subject information in tables.

• Present attendance percentages with progress bars.

• Include examination schedules and announcements.

• Make the layout responsive.

3. Assumptions

• Student details and marks are sample data.

• Attendance bars are visual indicators, not connected to a live database.

• This is a front-end prototype.

4. Algorithm

1. Start.

2. Create a sidebar and main dashboard area.

3. Add student profile and overview cards.

4. Create a table for semester results and subjects.

5. Display attendance using progress bars.

6. Add examination schedule and announcements.

7. Style and test the page at different widths.

8. Stop.

5. Flowchart

START → Create structure → Add content → Apply CSS → Test layout → Display webpage → END

6. Pseudocode

```
BEGIN
 CREATE EduPortal dashboard
 DISPLAY sidebar navigation
 DISPLAY student details and overview cards
 DISPLAY semester results table
 DISPLAY subject and attendance information
 DISPLAY exam schedule and announcements
 APPLY responsive styling
 TEST all sections
END
```

7. HTML/CSS Source Code

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>EduPortal</title>
<style>
body{margin:0;font-family:Arial;background:#f4f7fb;color:#20304a}
.layout{display:flex;min-height:100vh}
aside{background:#17365d;color:white;padding:20px;width:180px}
main{padding:22px;flex:1}.cards{display:flex;gap:12px;flex-wrap:wrap}
.card{background:white;padding:16px;border-radius:8px;flex:1 1 140px}
table{width:100%;border-collapse:collapse;background:white;margin:12px 0}
td,th{padding:9px;border:1px solid #ccd;text-align:left}
progress{width:80%}
@media(max-width:650px){.layout{display:block}aside{width:auto}}
</style></head><body><div class="layout">
<aside><h2>EduPortal</h2>Home<br>Results<br>Attendance<br>Subjects<br>Exams<br>Announcements</aside>
<main><h1>Student Dashboard</h1>
<p>Student: Varshini | Branch: AIML | Semester: 1</p>
<div class="cards"><div class="card"><h3>Semester GPA</h3>8.5</div>
<div class="card"><h3>Attendance</h3>92%</div><div class="card"><h3>Subjects</h3>6</div></div>
<h2>Semester Results</h2><table><tr><th>Subject</th><th>Marks</th><th>Result</th></tr>
<tr><td>Python</td><td>88</td><td>Pass</td></tr><tr><td>Mathematics</td><td>84</td><td>Pass</td></tr></table>
<h2>Subject-wise Attendance</h2><p>Python <progress value="92" max="100"></progress> 92%</p>
<p>Mathematics <progress value="86" max="100"></progress> 86%</p>
<h2>Examination Schedule</h2><table><tr><th>Exam</th><th>Date</th></tr><tr><td>Python</td><td>As announced</td></tr></table>
<h2>Announcements</h2><p>Check the notice board for updates.</p>
</main></div></body></html>
```

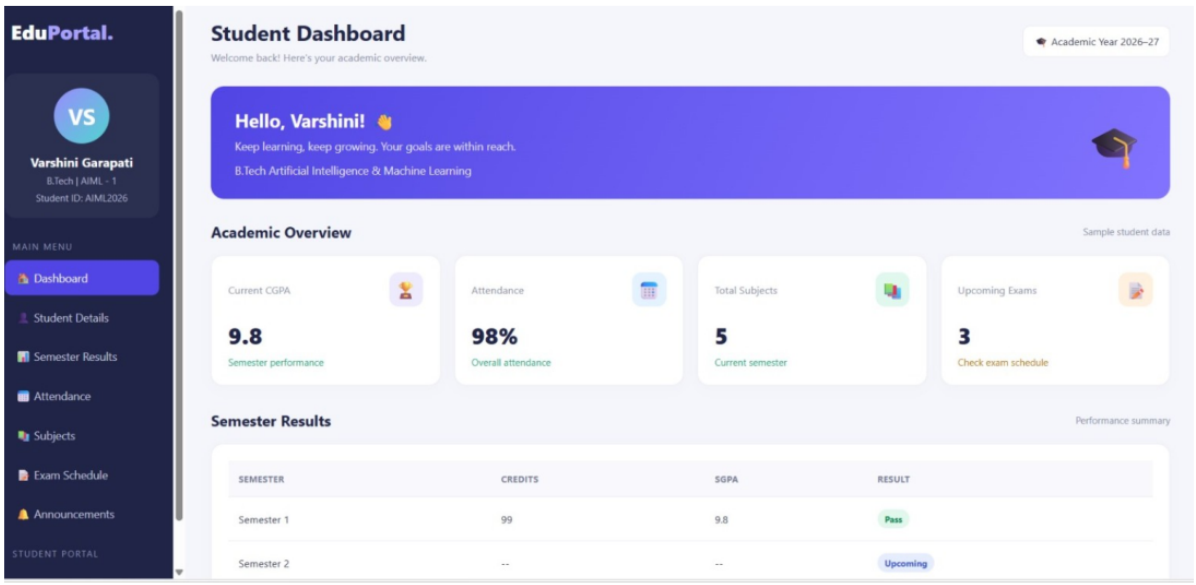
8. Test Cases

Test	Action / Check	Expected Result
1	Open dashboard	Sidebar and overview cards appear
2	Review results	Marks and result table appear
3	Review attendance	Percentages and progress bars appear
4	Check exam section	Schedule and announcements appear
5	Resize browser	Layout adapts to narrow screens

OUTPUT SCREENSHOT 1

ashboard – EduPortal

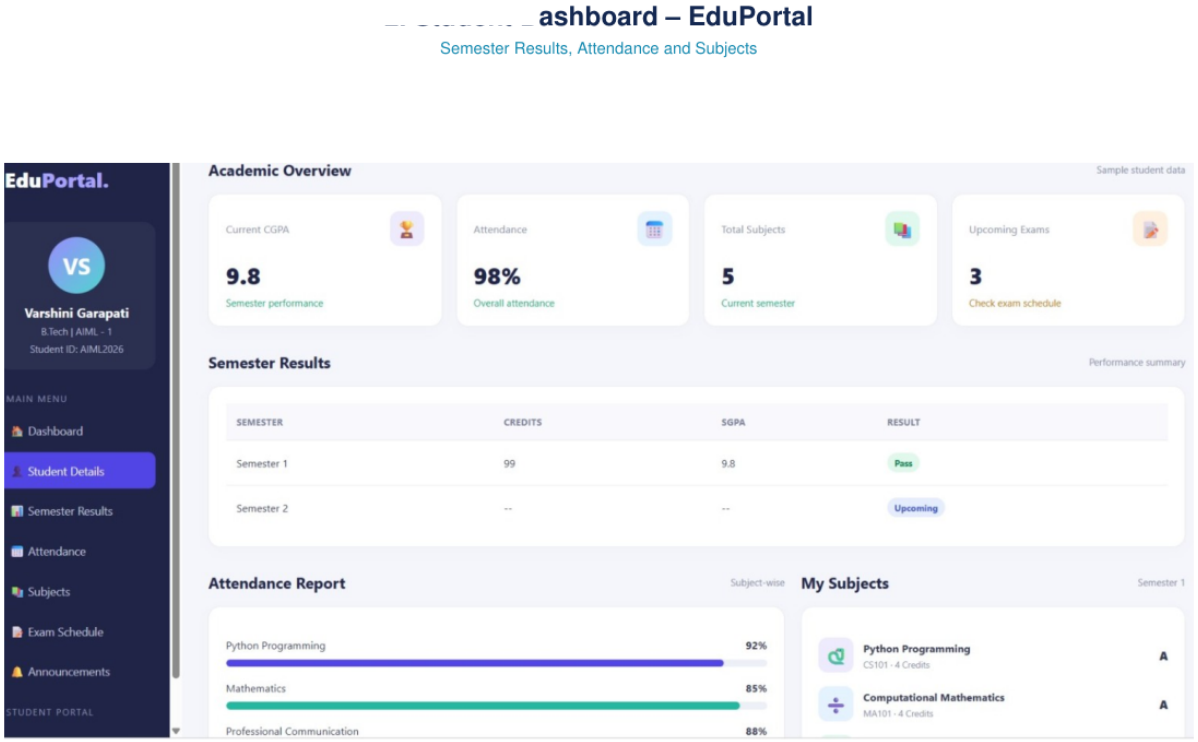
Dashboard and Academic Overview



Page

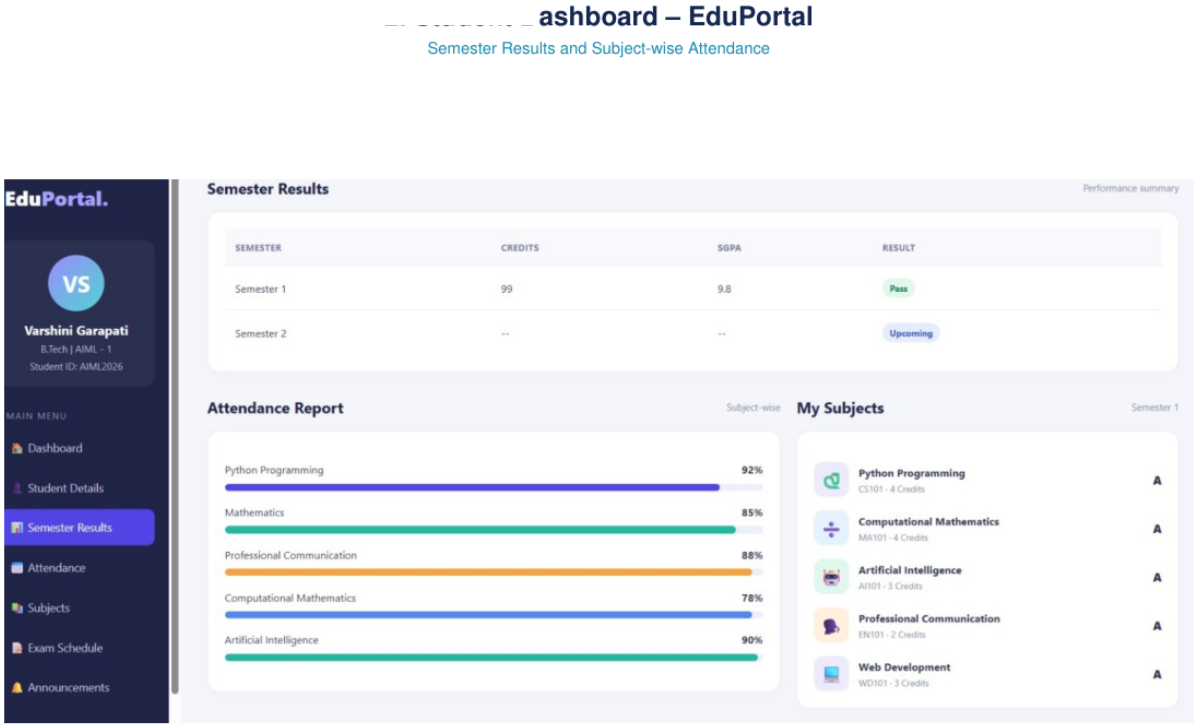
Results, Attendance and Subjects

OUTPUT SCREENSHOT 2



Page

OUTPUT SCREENSHOT 3

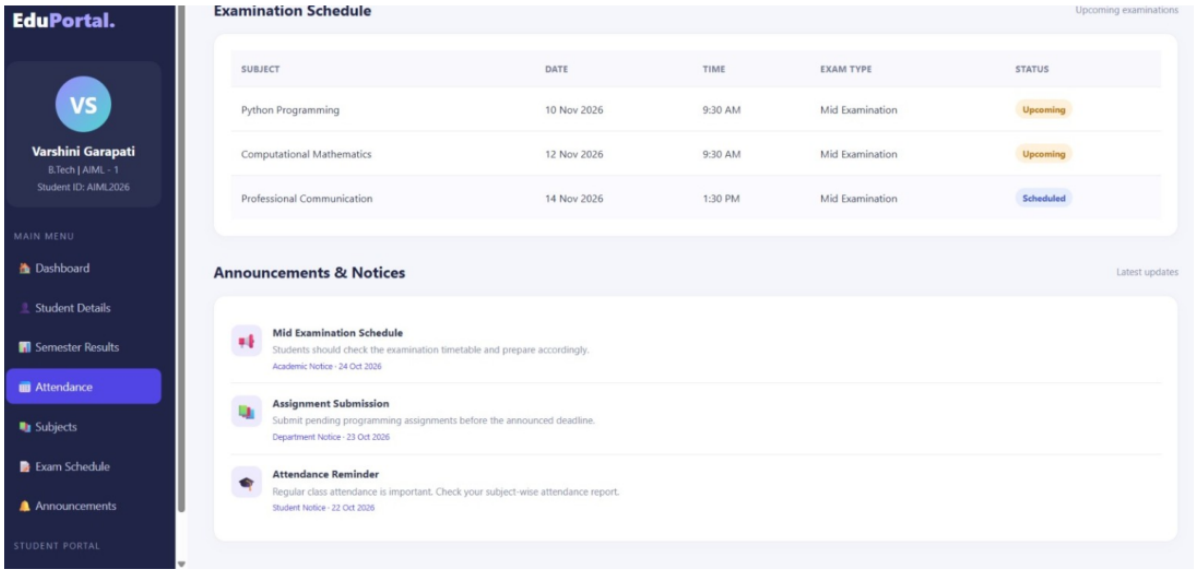


ion Schedule and Announcements

OUTPUT SCREENSHOT 4

ashboard – EduPortal

Examination Schedule and Announcements



Page

CASE STUDY 3: Doctor Appointment Website

1. Problem Statement

Design a doctor appointment website with doctor profiles, consultation slots, patient registration, and an appointment form. Patients should be able to review doctors and enter appointment details.

2. Objectives

• Display doctor profiles and specializations.

• Show consultation-slot information.

• Provide patient registration fields.

• Allow selection of doctor, date, and time.

• Use clear labels and responsive styling.

3. Assumptions

• The form is a front-end sample and does not send data to a hospital system.

• Doctor names and slots are illustrative.

• Real appointment confirmation requires a backend.

4. Algorithm

1. Start.

2. Create the page header and navigation.

3. Display doctor profile cards and consultation slots.

4. Create patient registration inputs.

5. Add doctor, date, and time fields to the appointment form.

6. Style the page and form controls.

7. Test required fields and responsive layout.

8. Stop.

5. Flowchart

START → Create structure → Add content → Apply CSS → Test layout → Display webpage → END

6. Pseudocode

```
BEGIN
 CREATE doctor appointment webpage
 DISPLAY doctor profiles and consultation slots
 DISPLAY patient registration form
 DISPLAY appointment form
 ALLOW doctor, date, and time selection
 APPLY CSS styling
 TEST form fields and page layout
END
```

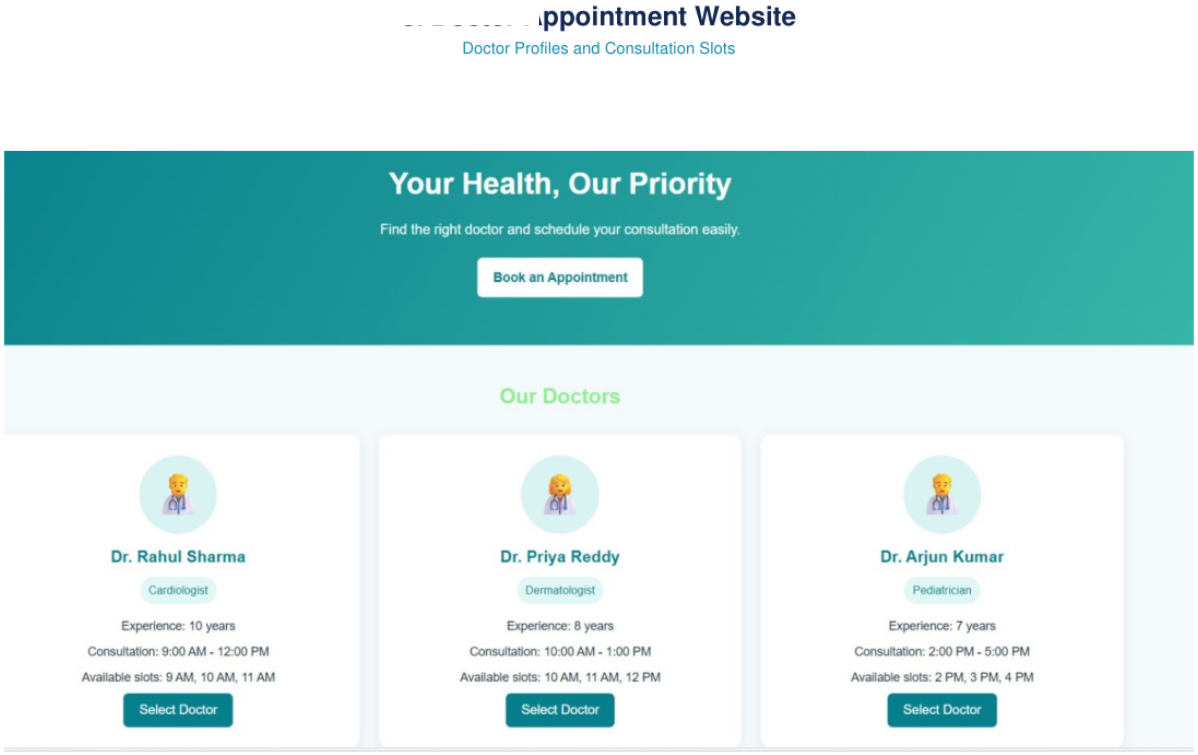
7. HTML/CSS Source Code

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Doctor Appointment</title>
<style>
body{font-family:Arial;margin:0;background:#f3fbfa;color:#173b3a}
header{background:#176b67;color:white;padding:20px}main{padding:22px}
.cards{display:flex;gap:14px;flex-wrap:wrap}
.card,form{background:white;padding:18px;border-radius:10px;flex:1 1 220px}
label{display:block;margin-top:10px}
input,select,button{box-sizing:border-box;width:100%;padding:9px;margin-top:4px}
button{background:#176b67;color:white;border:0;margin-top:14px}
</style></head><body><header><h1>CarePoint – Doctor Appointments</h1>
<p>Find a doctor and request a consultation</p></header><main>
<h2>Doctor Profiles and Consultation Slots</h2><section class="cards">
<article class="card"><h3>Dr. Ananya Rao</h3><p>General Physician</p><p>Slots: 10:00 AM, 11:30 AM</p></article>
<article class="card"><h3>Dr. Kiran Kumar</h3><p>Cardiologist</p><p>Slots: 2:00 PM, 4:00 PM</p></article></section>
<h2>Patient Registration and Appointment Form</h2><form>
<label>Patient Name</label><input type="text" required placeholder="Enter full name">
<label>Age</label><input type="number" min="1" required>
<label>Phone Number</label><input type="tel" required placeholder="Enter phone number">
<label>Select Doctor</label><select required><option value="">Choose a doctor</option>
<option>Dr. Ananya Rao</option><option>Dr. Kiran Kumar</option></select>
<label>Appointment Date</label><input type="date" required>
<label>Consultation Slot</label><select required><option value="">Choose a slot</option>
<option>10:00 AM</option><option>11:30 AM</option><option>2:00 PM</option><option>4:00 PM</option></select>
<button type="submit">Request Appointment</button></form></main></body></html>
```

8. Test Cases

Test	Action / Check	Expected Result
1	Open page	Doctor cards and slots appear
2	Enter patient details	Input fields accept entries
3	Select doctor and slot	Dropdown options can be selected
4	Leave required fields blank	Browser prompts for required fields
5	Resize browser	Cards and form remain readable

OUTPUT SCREENSHOT 1



Page

OUTPUT SCREENSHOT 2

